

Summer of Science 2026: Large Language Model (CS25)

Plane of Action

Mentee: Jithin K

Mentor: Sayandeep Halder

May 25, 2026

Timeline

- **Week 1 (May 25 - 30): Mathematical & Programming Foundations**
 - Since I only know basic neural networks, simple activation functions like ReLU, and standard calculus, I need to start here. I will brush up on the underlying math—especially matrix multiplication, partial derivatives, the chain rule, and concepts like cross-entropy loss and probability distributions. To get comfortable, I will write a simple Multi-Layer Perceptron (MLP) from scratch using only NumPy, manually coding the forward passes and the backpropagation gradients without using PyTorch's automatic shortcuts.
- **Week 2 (May 31 - June 6): Classical NLP & Pre-Transformer Sequence Models**
 - I will learn how text is actually turned into numbers that a neural network can read, moving from simple techniques to dense vectors. I will study older recurrent architectures like RNNs and LSTMs to see how they try to remember past words in a sentence, and learn why they struggle with long text due to vanishing gradients. For practice, I will build and train a basic character-level LSTM in PyTorch to see if I can get it to predict the next letter in a text dataset like Shakespeare.
- **Week 3 (June 7 - 13): The Transformer Architecture — The Cornerstone**
 - This week I will focus entirely on understanding the Transformer, which replaced LSTMs by looking at a whole sentence at once. I need to understand the "self-attention" mechanism—how query, key, and value matrices interact to find connections between words. I will try to implement a clean, simple decoder-only Transformer from scratch in PyTorch, manually handling the attention math, the masks that hide future words, and the position helpers.
- **Week 4 (June 14 - 20): The Pre-Training Paradigm — BERT, GPT & T5**
 - I will learn how large models are trained on massive amounts of raw internet text before being used for specific tasks. I want to see the difference between models that read left-to-right to generate text (like GPT) and models that look both ways to fill in missing words (like BERT). To see how this works in practice, I will use the Hugging Face library to take a ready-made BERT model and fine-tune it to classify text sentiments.

- **Week 5 (June 21 - 27): Scaling Laws, Modern Open-Source LLMs & Efficient Training**
 - I want to understand why simply making neural networks bigger makes them so much smarter, looking at the rules that govern compute and data size. I will explore how real-world models like LLaMA are put together, learning about minor tweaks to normalization and positions. I will write a script to load an open-source 7B or 8B model on my machine to see how much memory it uses and test how fast it generates words.

Mid-term Submission

- **Week 6 (June 28 - July 3): Instruction Tuning, RLHF, DPO & Alignment**
 - I will figure out how a raw model that just guesses the next word is turned into a helpful AI assistant that answers questions. I will read up on how models are trained on instruction examples and how they are tuned using human feedback or preference pairs. I will try running a small fine-tuning script using the [TRL](#) library to see how Direct Preference Optimization (DPO) changes a model's answers.
- **Week 7 (July 4 - 10): Parameter-Efficient Fine-Tuning, Quantization & On-Device LLMs**
 - Since giant networks take up too much GPU memory to train or run easily, I will study the engineering tricks used to shrink them. I will learn about quantization—squeezing weights from large floats down to tiny 4-bit integers—and LoRA, which keeps the main network frozen while training a tiny set of extra adapter weights. I will write a script to fine-tune an 8B model locally using these 4-bit QLoRA methods to keep memory low.
- **Week 8 (July 11 - 17): Retrieval-Augmented Generation, Agents & Prompt Engineering**
 - I will explore how to connect an LLM to external files so it can look up real facts instead of guessing. I'll learn how to turn text documents into vectors, store them in a searchable database, and inject the matching text right into the prompt. I will write code to parse some PDFs, save them into a local FAISS index, and build a simple chatbot that answers questions based on those files.
- **Week 9 (July 18 - 24): Vision, Multimodal Models & Beyond Text**
 - I want to see how the same transformer networks I used for text can be expanded to look at images. I will study how pictures are chopped up into small squares so a transformer can read them like words, and how vision and text models are glued together. I will write a pipeline using a multimodal model like LLaVA to feed an image tensor and a text prompt together into the network for visual question-answering.
- **Week 10 (July 25 - 31): Frontier Models, Evaluation, Safety & Research Frontiers**
 - In the final week, I will survey the most advanced reasoning models around today, like DeepSeek-R1 and OpenAI's o-series, to see how they use internal thinking steps. I will learn how researchers test these systems using standard test banks and explore basic safety principles. I will use a benchmarking tool

like `lm-evaluation-harness` to run math and logic tests on a couple of open models to see how they score.

Final Submission

References

- **Seminal Papers & Concepts:** "Attention Is All You Need", BERT, GPT-3, LLaMA, InstructGPT (RLHF), DPO, LoRA, RAG, ViT.
- **Key Resources:** Andrej Karpathy's "Let's build GPT" (YouTube), Jay Alammar's "The Illustrated Transformer" (Blog), Hugging Face TRL/PEFT/Transformers Documentation/Hubs.
- **Key Open-Source Repositories:** nanoGPT, LLaMA, transformers, trl, peft, vllm, llama.cpp, langchain, lm-evaluation-harness, diffusers.
- **Evaluation & Grading Rubric:** Criterion breakdown for Conceptual Understanding, Coding Proficiency, Paper Comprehension, and Research Thinking.
- **Supplementary Reference Architecture:** Conceptual evolution from RNN/LSTM through modern reasoning models (o1, DeepSeek-R1, Gemini 2.5).
-